How does violating homoscedasticity affect inference in linear regression?

Yanqi Huang, MS Student[1]

Abstract

Ordinary least squares (OLS) regression relies on the assumption of constant error variance. This simulation study shows that while OLS estimates remain unbiased under heteroscedasticity, model-based standard errors underestimate true variability, leading to confidence intervals with poor coverage. These results highlight the need to account for heteroscedasticity to ensure valid inference.

Introduction

Ordinary least squares (OLS) regression is widely used to explore relationships between explanatory factors and an outcome variable. For example, in biomedical research, it is often applied to assess associations between risk factors and health outcomes. The key assumptions of OLS include linearity, normality of errors, and homoscedasticity. The purpose of this study is to evaluate how varying levels of heteroscedasticity affect inference in OLS regression using simulation.

Motivating Problem

In practice, the assumption of homoscedasticity is frequently overlooked when fitting linear regression models. However, violations of this assumption can have a greater impact on the validity of regression results than violations of normality [1]. This simulation study will examine the impact of mild, moderate, and strong levels of heteroscedasticity on bias, model based standard error(SE), empirical standard error, their relative error and confidence interval (CI) coverage in ordinary OLS regression.

Simulation Set-Up

To evaluate how key performance metrics vary under increasing heteroscedasticity, the simulation was designed as follows:

1. The predictor x_i is generated from a uniform distribution $x_i \sim \text{Uniform}(1, 6)$ with a sample size of $n = 100$ [2].
2. The error variance is defined as a function of x, with parameter c controlling the degree of heteroscedasticity:

$$\varepsilon_i \sim \mathcal{N}\left(0,\ \frac{\exp\{c(x_i - \mu_x)\}}{\mathbb{E}[\exp\{c(x_i - \mu_x)\}]}\right)$$

   - $c = 0$ (homoscedastic baseline): The variance simplifies to 1 for all x_i, giving

$$\varepsilon_i \sim \mathcal{N}(0, 1)$$

   - $c > 0$: three levels of heteroscedasticity were examined: c=0.3 (mild), where variance increases only slightly with x_i; c=1 (moderate), where variance grows more noticeably across the range of x_i; c=6 (strong), where variance differs substantially with x_i.
3. The response variable is defined as

$$y_i = 14 + 0.4x_i + \varepsilon_i$$

   with the true slope $\beta_1 = 0.4$ [2].
4. Each scenario is replicated 1000 times, consistent with the average number of iterations used in previous studies [2].

Residual plots against fitted values in Figure 1 show that increasing c produces greater spread of residuals with $\hat{y}$, reflecting progressively stronger heteroscedasticity.

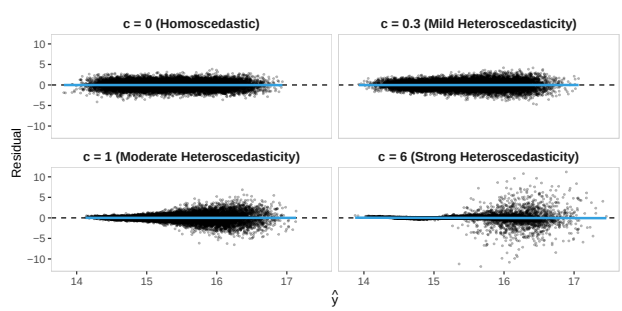

Figure 1: Residuals vs. fitted values for OLS regression under homoscedasticity(c=0) and increasing levels of heteroscedasticity(c=0.3,1,6)

To isolate heteroscedasticity itself, the variance function is normalized so that the average error variance $\mathbb{E}[\text{Var}(\varepsilon_i)]$ is fixed at 1 (Table 1, left panel). This adjustment keeps the noise level constant and allows fair

[1]Department of Biostatistics and Informatics, Colorado School of Public Health, University of Colorado Anschutz Medical Campus, Aurora, CO

comparison between homoscedastic and heteroscedastic settings. Without adjustment, both the average error variance and the variance of Y rise together, confounding heteroscedasticity with overall noise (Table 1, right panel)

Table 1: Variance of Y and expected error variance across heteroscedasticity levels (c)

	Normalized		Unnormalized	
c	$\mathrm{Var}(Y)$	$\mathbb{E}[\mathrm{Var}(\varepsilon \mid X)]$	$\mathrm{Var}(Y)$	$\mathbb{E}[\mathrm{Var}(\varepsilon \mid X)]$
0	1	1	1	1
0.3	1	1	1	1
1	1	1	3	2
6	1	1	110319	109623

Results

Figure 2 summarizes the effect of increasing heteroscedasticity(c) on the estimation of $\hat{\beta}_1$, with 95% Monte Carlo confidence intervals. Bias stayed near zero across all settings, confirming that OLS provided unbiased slope estimates. In contrast, variability measures diverged: the empirical SE (true variability across simulations) rose from 0.069 at c=0 to 0.113 at c=6, while the model based SE (average OLS reported SE) fell slightly from 0.069 to 0.065. As a result, the model based SE increasingly underestimated true variability, with relative error reaching –42.1% at c=6. Correspondingly, CI coverage declined from 94.6% under homoscedasticity to 73.4% under strong heteroscedasticity.

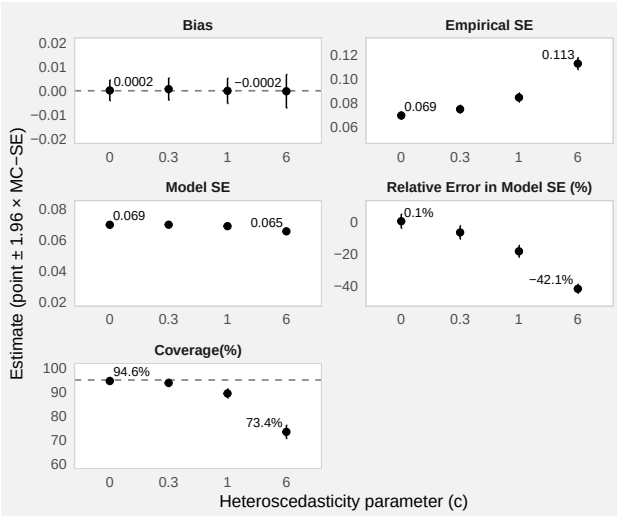

Figure 2: Metrics with 95% Monte Carlo confidence intervals across levels of heteroscedasticity

While Figure 2 illustrates the overall trends, Table 2 provides the corresponding numerical results with Monte Carlo standard errors, showing the precision of the simulation estimates. For example, bias was consistently negligible, with all estimates within ±0.004. at c=6 the empirical standard error was 0.113 (±0.003) compared to a model based standard error of 0.065 (±0.001) and coverage estimates within ±1.4.

Table 2: Simulation results for OLS regression under heteroscedasticity with Monte Carlo standard errors (in parentheses)

c	Bias	Model SE	Empirical SE	Relative error in model SE (%)	CI Coverage (%)
0	0.0002 (±0.002)	0.069 (±0.000)	0.069 (±0.002)	0.1 (±2.3)	94.6 (±0.7)
0.3	0.0007 (±0.002)	0.070 (±0.000)	0.075 (±0.002)	-6.9 (±2.1)	93.8 (±0.8)
1	-0.0000 (±0.003)	0.069 (±0.000)	0.084 (±0.002)	-18.7 (±1.8)	89.4 (±1.0)
6	-0.0002 (±0.004)	0.065 (±0.001)	0.113 (±0.003)	-42.1 (±1.4)	73.4 (±1.4)

Conclusion

These simulations show that OLS slope estimates remain unbiased under heteroscedasticity, but inference based on model based standard errors is unreliable. As heteroscedasticity increased, empirical variability rose while model based SEs declined slightly, resulting in confidence interval coverage falling up to 20 percentage points below nominal 95%. Thus, OLS-based inference gives overly optimistic precision, and robust alternatives should be preferred such as heteroscedasticity consistent SEs or alternative models.

This study is limited to one covariate distribution, a fixed sample size, and a single form of heteroscedasticity. Future work could examine other covariate distributions, larger samples, and alternative variance structures. Overall, careful attention to homoscedasticity assumption is essential, as violations can lead to misleadingly narrow intervals and incorrect conclusions.

References

1. Yang K, Tu J, Chen T. Homoscedasticity: an overlooked critical assumption for linear regression. General Psychiatry. 2019;32:e100148. https://doi.org/10.1136/gpsych-2019-100148
2. Saad, Marwa & Youssef, Ahmed & Abdel-Latif, Shereen. (2023). The Performance of Quantile Regression and Linear Regression with Heteroskedasticity was Compared in a Simulated Study. Journal of Statistics Applications & Probability. 12. 635-655. 10.18576/jsap/120226.

Code Appendix

```
knitr::opts_chunk$set(echo = TRUE)
## ================================
## 0. Load needed packages
## ================================
# Helper function: install if not already available
load_or_install <- function(pkg){
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
  library(pkg, character.only = TRUE)
}

# List of needed packages
packages <- c("dplyr", "ggplot2",  "kableExtra", "webshot2","magick","knitr","tidyverse","tidyr","ggh4x"
# Load or install each
lapply(packages, load_or_install)

## ================================
## 0.1 Setup
## ================================
seed <- 1234
sample_size <- 100
n_reps <- 1000
# true parameters of the linear model for data generation
beta0 <- 14
beta1 <- 0.4
# x is uniform distribution
x_min <- 1
x_max <- 6
# mean of the error term
eps_mean <- 0
# heteroscedasticity parameters to simulate
# when c=0, variance is 1 and it is homoscedastic baseline.
c_params <- c(0,0.3,1,6)
# store parameters in a list for easy passing to functions
params <- list(
  n = sample_size,
  x_min = x_min,
  x_max = x_max,
  eps_mean = eps_mean,
  beta0 = beta0,
  beta1 = beta1
```

```r
)
# path to save results
save_path <- "~/Library/CloudStorage/Dropbox/School/CU/fall 2025/BIOS 6618 adv biostatistical method/mid

## ================================
## 1. Function
## ================================
#-------------------------------------------------------------
#' Compute heteroscedastic error variance functions
#'
#' This function is designed for simulation studies where the predictor x
#' is assumed to be uniformly distributed on [a, b]. It defines the variance
#' of the error term as an exponential function of x, centered at the midpoint
#' of the interval, and scaled by the heteroscedasticity parameter c.
#'
#' The normalization factor is computed under the assumption that
#' X ~ Uniform[a, b], ensuring that the average normalized variance across
#' the uniform distribution equals 1.
#'
#' @param x Numeric vector. Predictor values at which to evaluate the variance.
#' @param c Numeric scalar. Heteroscedasticity strength parameter. Larger values
#'   of c make the variance more uneven across x.
#' @param a Numeric scalar. Lower bound of the covariate interval.
#' @param b Numeric scalar. Upper bound of the covariate interval.
#'
#' @return
#' A list with two components:
#' - eps_var_norm: Normalized variance function exp(c * (x - mu)) / norm,
#'   scaled so that the average variance over X ~ Uniform[a, b] is 1.
#' - eps_var_unnorm: Unnormalized variance function exp(c * (x - mu)).
get_eps_var <- function(x, c, a, b) {
  # midpoint of the interval
  mu <- (a + b) / 2

  # scaling term z
  z <- (b - a) * c / 2

  # Normalization factor = E[exp(c(X - mu))] under X ~ Unif[a,b]
  # = sinh(z)/z, with Taylor expansion for numerical stability when z ~ 0
  norm <- if (abs(z) < 1e-6) {
    1 + z^2/6 + z^4/120   # approximation of sinh(z)/z
  } else {
    sinh(z) / z
  }

  # Normalized variance: divides by norm so average variance = 1
  eps_var_norm <- exp(c * (x - mu)) / norm

  # Unnormalized variance: grows/shrinks with c, no normalization
  eps_var_unnorm <- exp(c * (x - mu))

  # Return both versions
  return(list(
    eps_var_norm   = eps_var_norm,
```

```r
    eps_var_unnorm = eps_var_unnorm
  ))
}

#-----------------------------------------------------------
#' Simulate linear regression data with controllable heteroscedasticity
#'
#' This function generates synthetic data for a linear regression model
#' @param c Numeric. Heteroscedasticity parameter controlling the degree of
#'   variance inflation as a function of x.
#' @param x Numeric vector of length n. Predictor values used directly in
#'   the simulation. Supplying the same x across different values of c
#'   allows apples-to-apples comparisons by holding the predictor
#'   distribution constant.
#' @param params A list containing:
#'   - n: sample size
#'   - x_min, x_max: range of x (uniform)
#'   - eps_mean: mean of the error term
#'   - beta0, beta1: regression coefficients
#' @param verbose Logical (default = FALSE). If TRUE, the function prints a
#'   short status message ("Data simulated successfully") along with a
#'   diagnostic table of parameters and summary statistics
#'
#' @return A data.frame with columns:
#'   - x: predictor values
#'   - y: response values
#'   - c_param: heteroscedasticity parameter
#'   - eps: simulated error terms
#'   - eps_var: conditional variance of each error term
#'   - y_unnorm: response values from unnormalized variance
#'   - eps_unnorm: error terms from unnormalized variance
#'   - eps_var_unnorm: unnormalized conditional variance of each error term
#'
#' @details
#' Why normalization is applied:
#'   The variance function is scaled so that the average error variance across x
#'   remains equal to 1 for all values of c. Without this normalization, larger c
#'   would both increase heteroscedasticity and inflate the overall noise level,
#'   confounding the comparison between homoscedastic and heteroscedastic settings.
#'   By holding the average variance constant, the simulation isolates the impact
#'   of heteroscedasticity on inference rather than on total noise.
#'
#' Why store extra parameters:
#'   Returning summary statistics (mean of eps_var, variance of y, etc.)
#'   provides a quick check that the simulation is working as expected,
#'   and documents the conditions under which the data were generated.
get_data <- function(c, x, params, verbose=FALSE) {
  n <- params$n
  x_min <- params$x_min
  x_max <- params$x_max
  eps_mean <- params$eps_mean
  beta0 <- params$beta0
  beta1 <- params$beta1
```

```r
  # Compute conditional error variance for each x
  eps_var_all <- get_eps_var(x, c, a=x_min, b=x_max)
  eps_var <- eps_var_all$eps_var_norm
  eps_var_unnorm <- eps_var_all$eps_var_unnorm

  # Draw error terms
  eps <- rnorm(n, mean = eps_mean, sd = sqrt(eps_var))
  eps_unnorm <- rnorm(n, mean=eps_mean, sd=sqrt(eps_var_unnorm))

  # Generate response variable
  y <- beta0 + beta1 * x + eps
  y_unnorm <- beta0 + beta1 * x + eps_unnorm

  # Return dataset
  data <- data.frame(
    x = x,
    y = y,
    c_param = c,   # renamed for clarity
    eps = eps,
    eps_var = eps_var,
    y_unnorm = y_unnorm,
    eps_unnorm = eps_unnorm,
    eps_var_unnorm = eps_var_unnorm
  )
  params = data.frame(
      c = c, n = n, x_min = x_min, x_max = x_max, x_bar=mean(x), sample_var_x=var(x),
      beta0 = beta0, beta1 = beta1, eps_mean = eps_mean, mean_eps_var = mean(eps_var), var_y=var(y)
    )
  if (verbose){
    message("Data simulated successfully")
    print(params)}
  return(data)
}
#-------------------------------------------------------------
#' Fit OLS and extract inference metrics for the slope coefficient
#'
#' This function fits a simple linear regression model of the form
#' $y = \beta_0 + \beta_1 x + \varepsilon$ using ordinary least squares.
#' It returns the estimated slope, its standard error, 95% confidence interval,
#' and p-value based on classical OLS assumptions.
#'
#' @param dat A data.frame containing the variables:
#'   - x: numeric predictor
#'   - y: numeric response
#'
#' @return A data.frame with one row containing:
#'   - beta1_hat: estimated slope coefficient for x
#'   - se: standard error of the slope estimate
#'   - ci_lower: lower bound of the 95% confidence interval
#'   - ci_upper: upper bound of the 95% confidence interval
#'   - p_value: p-value testing H0: beta1 = 0
#'   - n: sample size used in the regression
#'   - r_squared: model R-squared
#'   - c_param: heteroscedasticity parameter used in data generation
```

```r
#'   - beta1_true: true slope parameter used in data generation
#'   - beta0_true: true intercept parameter used in data generation
#'
#' @details
#' Why this is useful:
#'   These metrics allow assessment of bias, efficiency, coverage, and test size
#'   across different simulation settings. In particular:
#'   - beta1_hat can be compared to beta1_true to check bias.
#'   - se reflects estimation precision.
#'   - ci_lower and ci_upper allow CI coverage evaluation.
#'   - p_value is used to assess power.
#'
#' Confidence intervals are computed using R's built-in confint() function,
#' which relies on the t distribution with residual degrees of freedom.
fit_lm <- function(dat, verbose=FALSE) {
  stopifnot(all(c("x","y") %in% names(dat)))

  fit <- lm(y ~ x, data = dat)
  dat$y_hat  <- fitted(fit)     # or predict(fit)
  dat$resid  <- residuals(fit)  # dat$y - dat$y_hat
  n <- nobs(fit)
  r2 <- summary(fit)$r.squared
  beta1_hat <- coef(fit)[["x"]]

  summ <- summary(fit)$coefficients
  se <- summ["x", "Std. Error"]
  pval <- summ["x", "Pr(>|t|)"]

  # 95% CI for slope directly from confint()
  ci <- confint(fit, "x", level = 0.95)
  ci_lower <- ci[1]
  ci_upper <- ci[2]
  result <- data.frame(
    beta1_hat = beta1_hat,
    se = se,
    ci_lower = ci_lower,
    ci_upper = ci_upper,
    p_value = pval,
    n = n,
    r_squared = r2,
    c_param = unique(dat$c_param),
    beta1_true = params$beta1,
    beta0_true = params$beta0
  )
  if (verbose){
    message("model run successfully")
    print(result)
  }
  return(list(result=result, data=dat))
}
#-------------------------------------------------------------
#' Run simulation study over multiple heteroscedasticity values
#'
#' This function performs repeated simulations of linear regression
```

```r
#' under varying heteroscedasticity parameters c, collecting
#' both the simulated datasets and the fitted model results.
#'
#' For each repetition:
#' - A single covariate sample x_rep ~ Uniform(x_min, x_max) is generated.
#' - For each c in c_values, outcome data are generated via `get_data()`,
#'   with heteroscedastic error variance defined by c.
#' - A linear model is fitted using `fit_lm()`, and both the data
#'   and fit summaries are stored.
#'
#' Seeds are carefully managed:
#' - The covariate x_rep depends only on the repetition index r.
#' - The error term depends on both repetition and c, but not on the
#'   order of the grid.
#'
#' @param c_values Numeric vector. Values of the heteroscedasticity parameter c.
#' @param params List of parameters, typically including:
#'   - x_min: Lower bound of covariate support.
#'   - x_max: Upper bound of covariate support.
#'   - n: Sample size per dataset.
#' @param n_reps Integer. Number of independent simulation repetitions.
#' @param seed Integer or NULL. Master seed for reproducibility.
#'
#' @return
#' A list with two components:
#' - fits: Data frame of fitted regression results across all reps and c values.
#'   Includes repetition and seed tags.
#' - datasets: Data frame of all simulated datasets, annotated with repetition
#'   and seed identifiers.
#'
#' @details
#' Seed setup rationale:
#' - The covariate x_rep is seeded only by the repetition index r.
#'   This ensures that within each repetition, the same x values are used
#'   across all c, allowing direct comparisons between heteroscedasticity
#'   settings without extra randomness from covariates.
#' - The error term seed is constructed as a function of r and c:
#'     local_seed <- seed + r*100000 + round(c*1000)
#'   This guarantees:
#'   * Independence of error draws across both reps and c values.
#'   * Reproducibility that does not depend on the order of c_values
#'     or the grid length.
#' - Together, this structure provides stable and comparable simulations
#'   while preserving reproducibility across runs.
simulate_over_c <- function(c_values, params, n_reps, seed = NULL) {
  x_min <- params$x_min; x_max <- params$x_max; n <- params$n

  fits_all     <- NULL
  datasets_all <- NULL

  total_iters <- n_reps * length(c_values)

  # progress bar + reserve a second line for status text
  pb <- utils::txtProgressBar(min = 0, max = total_iters, style = 3)
```

```r
  on.exit(close(pb), add = TRUE)
  cat("\n")

  iter <- 0L

  for (r in seq_len(n_reps)) {
    # Seed for x (depends only on repetition, not c)
    set.seed(seed + r)
    # for each rep, generate x once. x is same across all c but different across reps
    x_rep <- runif(n, min = x_min, max = x_max)
    # for each c, generate data using same x_rep
    for (j in seq_along(c_values)) {
      c_val <- c_values[j]
      # Seed for error term (depends on rep + c, but not grid length/order)
      local_seed <- seed + r*100000 + as.integer(round(c_val*1000))
      set.seed(local_seed)

      dat <- get_data(c = c_val, x = x_rep, params = params)
      fit_results_all <- fit_lm(dat)
      fit <- fit_results_all$result
      dat <- fit_results_all$data

      # tag
      dat$rep <- r; dat$seed <- seed
      fit$rep <- r; fit$seed <- seed

      # append
      datasets_all <- rbind(datasets_all, dat)
      fits_all     <- rbind(fits_all, fit)

      iter <- iter + 1L

      # throttle progress bar updates to reduce overhead
      if (iter %% 10L == 0L || iter == total_iters) {
        utils::setTxtProgressBar(pb, iter)
      }

      # live status line (overwrites same line)
      cat(sprintf(
        "\rrep %d/%d | c %d/%d (c = %s) | iter %d/%d",
        r, n_reps, j, length(c_values), as.character(c_val), iter, total_iters
      ))
      flush.console()
    }
  }

  # finish with a fresh line
  cat("\n")

  list(fits = fits_all, datasets = datasets_all)
}

#-----------------------------------------------------------
#' Sanity check for simulation results
```

```r
#'
#' This function verifies that the simulation results produced by `simulate_over_c()`
#' behave as expected. It checks variation of `x`, `eps`, and `y` across replications
#' and consistency of `x` across different `c` values within each replication.
#'
#' Specifically, it performs three groups of checks:
#' \enumerate{
#'   \item \strong{Within each c}: verifies that `x`, `eps`, and `y` vary across
#'         replications (`rep`).
#'   \item \strong{Across c within each rep}: verifies that the vector of `x`
#'         values is identical for all `c` values within the same replication.
#'   \item \strong{Across c overall}: verifies that the variance of `eps` and
#'         the variance of `y` differ across `c` values (expected under heteroscedasticity).
#' }
#'
#' @param res A list returned by `simulate_over_c()`. Must contain an element
#'   `datasets`, which is a data frame with columns `x`, `eps`, `y`, `c`, and `rep`.
#'
#' @return Prints a summary of the checks to the console.
#'
#' @details
#' - If all checks return `TRUE`, it indicates that the simulation logic is functioning
#' - If any check returns `FALSE`, it may indicate a problem in the simulation logic
#'   (e.g., seed placement, incorrect reuse of `x`, or missing heteroscedasticity).
#'
sanity_check <- function(res) {
  dat <- res$datasets

  # --- A) varies across reps within each c ---
  vary_by_rep <- function(vec) {
    # different group means across reps variation
    all(tapply(vec, list(dat$c, dat$rep), mean) |>
          apply(1, function(v) length(unique(v)) > 1))
  }
  x_varies_within_c   <- vary_by_rep(dat$x)
  eps_varies_within_c <- vary_by_rep(dat$eps)
  y_varies_within_c   <- vary_by_rep(dat$y)

  # --- B) for each rep, x is identical across all c (vector-wise) ---
  x_identical_across_c_per_rep <- {
    by_rep <- split(dat, dat$rep)
    all(vapply(by_rep, function(df_rep) {
      xs <- split(df_rep$x, df_rep$c)
      ref <- xs[[1]]
      # compare element-wise identity to the first c
      all(vapply(xs[-1], function(v) identical(as.numeric(v), as.numeric(ref)), logical(1)))
    }, logical(1)))
  }


  # --- C) across c: eps/y variance differ (expected if heteroscedastic by c) ---
  eps_var_diff_across_c <- length(unique(tapply(dat$eps, dat$c, var))) > 1
  y_var_diff_across_c   <- length(unique(tapply(dat$y,   dat$c, var))) > 1
```

```r
  cat("Sanity check:\n")
  cat("  x varies across reps within each c:            ", x_varies_within_c,   "\n")
  cat("  eps varies across reps within each c:          ", eps_varies_within_c, "\n")
  cat("  y varies across reps within each c:            ", y_varies_within_c,   "\n")
  cat("  x identical across c for each rep (elementwise): ", x_identical_across_c_per_rep, "\n")
  cat("  eps variance differs across c:                 ", eps_var_diff_across_c, "\n")
  cat("  y variance differs across c:                   ", y_var_diff_across_c,   "\n")
}

## ================================
## 2. Data Generation & Simulation Loop
## ================================
res <- simulate_over_c(c_values = c_params, params = params, n_reps = n_reps, seed=seed)

## ================================
## 3. Sanity Check
## ================================
sanity_check(res)

## ================================
## 4. Save Results
## ================================
if (!dir.exists(save_path)) {
  dir.create(save_path, recursive = TRUE)
}

# ISO 8601 date (date only):
iso_date <- format(Sys.Date(), "%Y-%m-%d")

# describe the run
experiment_tag <- "ols-heteroscedasticity-simulation"

# pull key params
n        <- unique(res$fits$n)
n_reps   <- max(res$fits$rep)
seed_val_input <- unique(res$fits$seed)
seed_val <- if (is.null(seed_val_input)) "na" else as.character(seed_val_input)
# get unique c values from fits
c_vals <- sort(unique(res$fits$c_param))
# collapse into compact string

# compact c-values for filenames: 0, 0.5, 1, 2 -> "c0-0.5-1-2"
c_slug <- paste0("c", paste(c_vals, collapse = "-"))

# build a consistent suffix with key params
param_suffix <- sprintf("n%d_reps%d_seed%s_%s", n, n_reps, seed_val, c_slug)

# final filenames
fits_rds      <- file.path(save_path, sprintf("%s_optimized-seed-best-param-centered-log-linear-fits_%s_
datasets_rds  <- file.path(save_path, sprintf("%s_optimized-seed-best-param-centered-log-linear-datasets

# save (RDS for fidelity + CSV for sharing)
saveRDS(res$fits,     fits_rds)
saveRDS(res$datasets, datasets_rds)
```

```r
# confirmation message
message("Results saved successfully:\n",
        "  - ", fits_rds, "\n",
        "  - ", datasets_rds, "\n")

## ================================
## 5. Summarize results
## ================================
# Summarize datasets to check variances
fits <- readRDS(fits_rds)
simulated_data <- readRDS(datasets_rds)
true_beta <- beta1
average_error_variance <- simulated_data %>%
  group_by(c_param) %>%
  summarise(
    var_y = round(var(y),0),                        # variance of Y
    mean_eps_var = round(mean(eps_var),0),          # average
    var_y_unnorm = round(var(y_unnorm),0),
    mean_eps_var_unnorm = round(mean(eps_var_unnorm), 0),
    .groups = "drop"
  )
average_error_variance <- average_error_variance %>%
  mutate(c_param = ifelse(c_param %% 1 == 0,
                          as.character(as.integer(c_param)),
                          as.character(c_param)))
# Summarize simulation results with MC-SEs
fits_summ <- fits %>%
  mutate(
    covered   = as.integer(true_beta >= ci_lower & true_beta <= ci_upper),
    rejected  = as.integer(p_value <= 0.05),
    est_err   = beta1_hat - true_beta
  ) %>%
  group_by(c_param) %>%
  summarise(
    R = n(),  # number of replications per c

    # Bias and its MC-SE
    bias         = mean(est_err, na.rm = TRUE),
    mcse_bias    = sd(est_err, na.rm = TRUE) / sqrt(R),

    # SD of beta1_hat across reps, with delta-method MC-SE: sd / sqrt(2*(R-1))
    sd_beta1_hat = sd(beta1_hat, na.rm = TRUE),
    mcse_sd_beta1 = sd_beta1_hat / sqrt(2 * pmax(R - 1, 1)),

    # Mean (reported) SE and its MC-SE
    se_beta1_hat = mean(se, na.rm = TRUE),
    mcse_se_mean = sd(se, na.rm = TRUE) / sqrt(R),

    # Relative error of model SE vs empirical SD
    rel_err_se    = (se_beta1_hat - sd_beta1_hat) / sd_beta1_hat,

    # --- MCSE for rel_err_se via delta method ---
    # Notation: M = mean(se), S = sd(beta1_hat)
    # g(M, S) = M/S - 1; grad g = (1/S, -M/S^2)
```

```r
    varM          = var(se, na.rm = TRUE) / R,
    varS          = (sd_beta1_hat^2) / (2 * pmax(R - 1, 1)),
    mcse_rel_err  = sqrt( (1 / sd_beta1_hat)^2 * varM +
                          (se_beta1_hat^2 / sd_beta1_hat^4) * varS ),
    # Coverage and its MC-SE
    ci_coverage  = mean(covered, na.rm = TRUE),
    mcse_cov     = sqrt(ci_coverage * (1 - ci_coverage) / R),



    .groups = "drop"
  )

## ================================
## 6. Create figures
## ================================
# Create table with formatted metrics including MC-SEs
## 1) Find rows using numeric values from fits_summ
row_min_relerr <- which.min(fits_summ$rel_err_se)      # rel_err_se is in proportion (not %)
row_min_cov    <- which.min(fits_summ$ci_coverage)     # proportion (0-1)
row_max_empse  <- which.max(fits_summ$sd_beta1_hat)

## 2) Build the display table (your code)
fits_table <- fits_summ %>%
  mutate(
    c_param       = ifelse(c_param %% 1 == 0,
                           as.character(as.integer(c_param)),
                           as.character(c_param)),
    bias          = sprintf("\\makecell{%.4f \\\\ (±%.3f)}", bias, mcse_bias),
    se_beta1_hat  = sprintf("\\makecell{%.3f \\\\ (±%.3f)}", se_beta1_hat, mcse_se_mean),
    sd_beta1_hat  = sprintf("\\makecell{%.3f \\\\ (±%.3f)}", sd_beta1_hat, mcse_sd_beta1),
    rel_err_se    = sprintf("\\makecell{%.1f \\\\ (±%.1f)}", rel_err_se*100, mcse_rel_err*100),
    ci_coverage   = sprintf("\\makecell{%.1f \\\\ (±%.1f)}", ci_coverage*100, mcse_cov*100)
  ) %>%
  select(c_param, bias, se_beta1_hat, sd_beta1_hat, rel_err_se, ci_coverage)
my_labels <- c(
  "0" = "c = 0 (Homoscedastic)",
  "0.3" = "c = 0.3 (Mild Heteroscedasticity)",
  "1" = "c = 1 (Moderate Heteroscedasticity)",
  "6" = "c = 6 (Strong Heteroscedasticity)"
)

ggplot(simulated_data %>% slice_sample(n = 48000),
  aes(y_hat, resid)) +
  geom_point(alpha = 0.3, size = 0.4) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x, color = "#33A1E0") +
  facet_wrap(
    ~ c_param,
    ncol = 2,
    labeller = labeller(c_param = as_labeller(my_labels)),
    scales = "fixed"
  ) +
  labs(x = expression(hat(y)), y = "Residual") +
```

```r
  theme_bw(base_size = 12) +
  theme(
    panel.grid = element_blank(),            # no grid lines
    strip.background = element_blank(),      # no gray background for facet labels
    strip.text = element_text(size=12,face = "bold"),
    panel.border = element_rect(color = "lightgray", fill = NA, linewidth = 0.5) # light gray box
  )
average_error_variance |>
  kbl(
    booktabs = TRUE,
    escape   = FALSE,
    digits   = c(0, 3, 3, 3, 3),
    col.names = c(
      "$c$",
      "$\\mathrm{Var}(Y)$",
      "$\\mathbb{E}[\\mathrm{Var}(\\varepsilon\\mid X)]$",
      "$\\mathrm{Var}(Y)$",
      "$\\mathbb{E}[\\mathrm{Var}(\\varepsilon\\mid X)]$"
    ),
    caption = "Variance of $Y$ and expected error variance across heteroscedasticity levels ($c$)"
  ) |>
  add_header_above(c(" " = 1, "Normalized" = 2, "Unnormalized" = 2)) |>
  kable_styling(
    latex_options = c("HOLD_position","scale_down"),
    full_width    = FALSE,
    position      = "center",
    font_size     = 7
  )


metrics_long <- bind_rows(
  fits_summ %>% transmute(c_param, metric = "Bias",                 est = bias,             mcse = mcse_
  fits_summ %>% transmute(c_param, metric = "Empirical SE",         est = sd_beta1_hat,     mcse = mcse_
  fits_summ %>% transmute(c_param, metric = "Model SE",             est = se_beta1_hat,     mcse = mcse_
  fits_summ %>% transmute(c_param, metric = "Relative Error in Model SE (%)",est = round(rel_err_se*100,
  fits_summ %>% transmute(c_param, metric = "Coverage(%)",          est = ci_coverage*100,  mcse = mcse_
) %>%
  mutate(
    lo = est - 1.96 * mcse,
    hi = est + 1.96 * mcse,
    ref = case_when(
      metric == "Bias"        ~ 0,
      metric == "Coverage(%)" ~ 95,
      TRUE ~ NA_real_
    ),
    metric  = factor(metric, levels = c("Bias","Empirical SE","Model SE","Relative Error in Model SE (%)
    c_factor = factor(c_param, levels = c_params)
  )

# Make sure c_param is treated as factor with correct order
metrics_long$c_param <- factor(
  metrics_long$c_param,
  levels = sort(unique(metrics_long$c_param))
)
```

```r
# Labels only for c=0 and c=6
label_data <- metrics_long %>%
  filter(c_param %in% c("0", "6")) %>%
  mutate(
    metric_chr = as.character(metric),
    label_val = case_when(
      grepl("%", metric_chr) ~ sprintf("%.1f%%", est),   # 1 decimal + %
      metric_chr == "Bias"   ~ sprintf("%.4f", est),     # 4 decimals for Bias
      TRUE                   ~ sprintf("%.3f", est)      # otherwise 3 decimals
    ),
    hjust_val = ifelse(c_param == "0", -0.1, 1.1),
    vjust_val = -0.5
  )


p <- ggplot(metrics_long, aes(x = c_param, y = est)) +
  geom_hline(aes(yintercept = ref), linetype = "dashed", na.rm = TRUE, color = "grey50") +
  geom_errorbar(aes(ymin = lo, ymax = hi), width = 0, linewidth = 0.5, color = "black") +
  geom_point(size = 2, color = "black") +
  geom_text(
    data = label_data,
    aes(label = label_val, hjust = hjust_val, vjust = vjust_val),
    size = 3, color = "black"
  ) +
  ggh4x::facet_wrap2(~ metric, scales = "free_y", ncol = 2, axes = "all") +
  # facet-specific y-limit for Model SE
  ggh4x::facetted_pos_scales(
    y = list(
      metric == "Model SE" ~ scale_y_continuous(limits = c(0.02, 0.082)),
      metric == "Bias" ~ scale_y_continuous(limits = c(-0.02, 0.02)),
      metric == "Coverage(%)" ~ scale_y_continuous(limits = c(60, 100)),
      metric == "Empirical SE" ~ scale_y_continuous(limits = c(0.05, 0.13)),
      metric == "Relative Error in Model SE (%)" ~ scale_y_continuous(limits = c(10, -50))
    )
  ) +
  labs(
    x = "Heteroscedasticity parameter (c)",
    y = "Estimate (point ± 1.96 × MC-SE)"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.border = element_rect(color = "lightgray", fill = NA, linewidth = 0.5),
    panel.background = element_rect(fill = "white", color = NA),     # inside each panel
    plot.background  = element_rect(fill = "gray95", color = NA),
    strip.text = element_text(face = "bold", size = 10),            # bold facet titles
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank()
  )

p

tbl2 <- kbl(
  fits_table,
```

```r
  caption   = "Simulation results for OLS regression under heteroscedasticity with Monte Carlo standard
  booktabs  = TRUE,
  align     = "cc",
  col.names = c(
    "c",
    "Bias",
    "\\makecell{Model\\\\SE}",
    "\\makecell{Empirical\\\\SE}",
    "\\makecell{Relative\\\\error in\\\\model SE\\\\(\\%)}",
    "\\makecell{CI\\\\Coverage\\\\(\\%)}"
  ),
  escape = FALSE   # allow LaTeX code inside names
) %>%
  kable_styling(
    latex_options = c("HOLD_position"),
    full_width    = FALSE,
    position      = "center",
    font_size     = 9
  )
tbl2
```